TD : organiser la validation avec une Form Request

Objectif

Vous allez déplacer les règles de validation déjà présentes dans `DocumentController` vers une classe `DocumentRequest`. Les règles restent identiques : le titre et la catégorie sont obligatoires, de type texte et limités respectivement à 100 et 50 caractères. L'API conserve les mêmes routes et les mêmes réponses.

Durée indicative : 20 minutes de pratique, après la présentation de la notion.

Avant de commencer

- Vous avez terminé le TD API CRUD des documents et disposez du projet Laravel avec Docker Desktop et WSL 2.
- Ouvrez le dossier `app-laravel-postgresql` dans VS Code : il contient `artisan` et `compose.yaml`. Ouvrez un terminal PowerShell dans ce dossier.
- Partez de votre travail terminé au jour 2 ou de la branche **`jour2-api-crud-documents`**. La branche **`jour3-form-requests`** contient la correction de ce TD.
- Pour les essais avec `curl.exe`, la table `documents` doit exister dans la base de formation. Si ce n'est pas le cas, terminez d'abord le TD sur les migrations au moment prévu par le formateur.

Vérifiez la branche et les services :

```
git branch --show-current
docker compose ps
```

Les services `app` et `db` doivent être démarrés. Pour ouvrir la branche de départ si nécessaire, sauvegardez d'abord votre travail, puis utilisez `git switch jour2-api-crud-documents`. Si Git refuse le changement, ne forcez rien : demandez au formateur comment conserver vos modifications. Si la branche manque dans votre copie, `git fetch origin` peut actualiser les références distantes avant de réessayer.

1. Comprendre le changement

Actuellement, le contrôleur répète `$request->validate(...)` dans `store` et `update`. Une **Form Request** est une classe Laravel dédiée à la validation d'une requête. Nous allons y placer les mêmes règles une seule fois.

Laravel exécute `authorize()` puis `rules()` avant la méthode du contrôleur. Ici, `authorize()` renvoie `true` pour conserver le comportement actuel : cette étape ne met pas encore en place d'authentification ni de permissions. Si les données sont invalides, Laravel répond **422** à une requête qui attend du JSON ; sinon le contrôleur récupère les valeurs validées avec `$request->validated()`.

2. Créer la Form Request

Dans le terminal du projet, lancez :

```
docker compose exec app php artisan make:request DocumentRequest --no-interaction
```

Laravel crée `app/Http/Requests/DocumentRequest.php`. Si ce fichier existe déjà, vérifiez votre branche et demandez l'aide du formateur avant de le remplacer.

Dans VS Code, ouvrez **`app/Http/Requests/DocumentRequest.php`**, remplacez tout son contenu par le code suivant et enregistrez :

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class DocumentRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    /**
     * @return array<string, array<int, string>>
     */
    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:100'],
            'category' => ['required', 'string', 'max:50'],
        ];
    }
}
```

`required` impose une valeur, `string` attend du texte et `max` limite sa longueur. Ces règles sont les mêmes que celles du jour 2. Le bloc `@return` décrit le type du tableau pour les outils de développement.

3. Adapter le contrôleur

Ouvrez **`app/Http/Controllers/DocumentController.php`**. Remplacez tout son contenu par le code suivant, puis enregistrez :

```
<?php

namespace App\Http\Controllers;

use App\Http\Requests\DocumentRequest;
use App\Models\Document;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Response;

class DocumentController extends Controller
{
    public function index(): JsonResponse
    {
        return response()->json(Document::all());
    }

    public function show(Document $document): JsonResponse
    {
        return response()->json($document);
```

```
    }

    public function store(DocumentRequest $request): JsonResponse
    {
        $data = $request->validated();

        $document = Document::create($data);

        return response()->json($document, 201);
    }

    public function update(DocumentRequest $request, Document $document): JsonResponse
    {
        $data = $request->validated();

        $document->update($data);

        return response()->json($document);
    }

    public function destroy(Document $document): Response
    {
        $document->delete();

        return response()->noContent();
    }
}
```

L'import `use App\Http\Requests\DocumentRequest;` rend la classe disponible dans le contrôleur. Les méthodes `store` et `update` reçoivent désormais cette requête et récupèrent les données avec `validated()`. Les routes, les autres méthodes et les règles métier ne changent pas.

4. Vérifier les données acceptées et refusées

Dans VS Code, créez à la racine du projet un fichier `document-form-request.json` contenant :

```
{
  "title": "Contrat de formation",
  "category": "Administratif"
}
```

Créez ensuite un document :

```
curl.exe -i -X POST -H "Accept: application/json" -H "Content-Type: application/json" --data-binary "@document-form-request.json" http://localhost:8088/api/documents
```

Résultat attendu : `201 Created` et un objet JSON contenant le titre, la catégorie et un `id`. Notez cet identifiant : il dépend des données déjà présentes dans votre base. Relancer cette commande crée un autre document.

Créez à la racine du projet un second fichier `document-form-request-invalide.json` contenant :

```
{}
```

Testez la création invalide :

```
curl.exe -i -X POST -H "Accept: application/json" -H "Content-Type: application/json" --data-binary "@document-form-
```

```
request-invalide.json" http://localhost:8088/api/documents
```

Résultat attendu : `422 Unprocessable Content` et des erreurs JSON pour `title` et `category`. Les messages peuvent être en anglais. L'en-tête `Accept` demande une réponse JSON, y compris en cas d'erreur.

Testez aussi la modification avec les données invalides, en remplaçant `1` par l'identifiant noté après la création :

```
curl.exe -i -X PUT -H "Accept: application/json" -H "Content-Type: application/json" --data-binary "@document-form-request-invalide.json" http://localhost:8088/api/documents/1
curl.exe -i -H "Accept: application/json" http://localhost:8088/api/documents/1
```

Résultat attendu : le `PUT` répond `422`. Le `GET` répond ensuite `200` et affiche toujours le titre et la catégorie d'origine : la modification invalide n'a pas été enregistrée.

5. Vérifier les tests existants

```
docker compose exec app php artisan test --compact tests/Feature/DocumentApiTest.php
```

Résultat attendu : les quatre tests de l'API réussissent. Ils vérifient notamment la création, la modification, les erreurs 422 et l'absence de modification après un envoi invalide. La configuration de test utilise SQLite en mémoire : ces tests ne migrent pas la base PostgreSQL de formation.

6. Comparer avec la correction

La branche `jour3-form-requests` contient la correction. Sauvegardez votre travail avant tout changement de branche. Pour comparer sans changer de branche, après récupération éventuelle des références distantes :

```
git fetch origin
git diff origin/jour3-form-requests -- app/Http/Requests/DocumentRequest.php app/Http/Controllers/DocumentController.php
```

Un fichier créé mais pas encore suivi par Git peut ne pas apparaître dans cette comparaison : ouvrez aussi la correction avec `git show origin/jour3-form-requests:app/Http/Requests/DocumentRequest.php`.

Questions de compréhension

1. Pourquoi avons-nous déplacé les règles hors du contrôleur ?
2. Que fait Laravel si `title` est absent dans une requête JSON ?
3. Pourquoi `authorize()` renvoie-t-il `true` à cette étape ?
4. Que renvoie `$request->validated()` au contrôleur ?

En cas de difficulté

Situation	Vérification
`403 Forbidden`	Dans `DocumentRequest`, `authorize()` renvoie-t-il bien `true` ?
Classe `DocumentRequest` introuvable	Vérifiez le chemin, le namespace et l'import dans le contrôleur.
Réponse `422` inattendue	Vérifiez les noms `title` et `category`, les valeurs et leur longueur.
Erreur HTML au lieu de JSON	Ajoutez l'en-tête `Accept: application/json`.
`404` sur le `PUT`	Remplacez `1` par l'identifiant réellement créé.
Table absente ou conteneur arrêté	Vérifiez `docker compose ps` et terminez le TD sur les migrations selon les consignes du formateur.